

GPU Implementation of the PSO Algorithm

Nađa Kovačević
Computing and Informatics
Faculty of Electrical Engineering
University of Sarajevo
nkovacevic3@etf.unsa.ba

Nejra Pirija
Computing and Informatics
Faculty of Electrical Engineering
University of Sarajevo
npjirija2@etf.unsa.ba

Una Hodžić
Computing and Informatics
Faculty of Electrical Engineering
University of Sarajevo
uhodzic1@etf.unsa.ba

Abstract—This paper investigates the optimization of the Particle Swarm Optimization (PSO) algorithm through GPU-based implementation using the CUDA framework. PSO is a population-based metaheuristic inspired by collective behaviors observed in nature and is widely used to solve global optimization problems. When executed on CPUs, conventional PSO implementations suffer from performance limitations, particularly in cases involving large swarms and high-dimensional search spaces. Leveraging the massive parallelism of GPUs allows PSO to be executed more efficiently, leading to substantial reductions in computation time. The optimized implementations presented in this paper were evaluated using standard benchmark functions, demonstrating performance improvements over CPU-based versions for large-scale problems. This work focuses on enhancing a baseline PSO implementation and provides a comprehensive performance analysis of the achieved optimizations.

Index Terms—CUDA, Graphics Processing Units (GPU), Particle Swarm Optimization (PSO), Parallel Programming, SPSO Algorithm.

I. INTRODUCTION

Particle Swarm Optimization (PSO) represents a stochastic computational technique inspired by the collective intelligence observed in social groups, such as bird flocks. Rather than using traditional evolutionary operators, this method utilizes a population of candidate solutions, termed particles, which "fly" through a multidimensional search space. Each particle dynamically updates its trajectory by balancing its own historical success (*pbest*) with the best performance achieved by the entire swarm (*gbest*). This collaborative exchange of information allows the system to efficiently explore complex surfaces and converge toward global optima [1].

Historically, the algorithm evolved from basic social simulations to a powerful optimizer by stripping away unnecessary biological complexities. The initial version focused on velocity matching, but it was soon discovered that the interaction between individual memory and group knowledge was sufficient for optimization. To further enhance stability and control the balance between global exploration and local exploitation, researchers introduced the inertia weight [2]. This modification allowed for a better regulation of the particle's momentum, preventing the swarm from overshooting potential solutions too aggressively.

As the field matured, mathematical analyses provided deeper insights into particle trajectories. The introduction of the constriction factor [3] offered a theoretical guarantee of convergence. Today, PSO is recognized as a versatile tool

capable of solving a wide range of problems. Common application domains include continuous function optimization, training of artificial neural networks, feature selection, clustering, scheduling, and control systems. In engineering, PSO is frequently used for parameter tuning, structural optimization, and power system optimization, while in data science it has shown strong performance in machine learning and pattern recognition tasks. Its derivative-free nature and robustness against noisy and non-linear objective functions make PSO particularly attractive for problems where gradient information is unavailable or expensive to compute [4], [5].

Beyond basic acceleration, contemporary research emphasizes the critical importance of managing the GPU's memory hierarchy to minimize information exchange latency between particles. Paper [6] highlights that the strategic mapping of data into shared memory and the systematic reduction of atomic operation contention are decisive factors for achieving maximum throughput in swarms with high population densities. Paper [7] argues that the specific architectural characteristics of CUDA cores in server-grade accelerators directly influence the stability and convergence speed of asynchronous optimization implementations.

The focus of our project is on the development and optimization of a parallel implementation of the PSO algorithm using graphics processing units (GPUs). To further improve computational performance, we leverage NVIDIA's CUDA (Compute Unified Device Architecture), which enables massive parallel execution of particle updates. This approach can significantly reduce runtime and has been shown to achieve substantial speedups for high-dimensional optimization problems [8]. Drawing on the fundamental concepts of a previously implemented sequential version and analyzing relevant scientific papers, we apply the CUDA architecture to leverage the massive parallelism of the GPU. The final goal of this work is a detailed performance comparison of different implementations, the identification of speedup, and an analysis of algorithmic efficiency depending on the hardware platform.

II. LITERATURE REVIEW

The work by Hussain, Hattori, and Fujimoto [9] deals with an efficient parallel implementation of the Standard Particle Swarm Optimization (SPSO) using GPU architecture and the CUDA programming model. It starts from the fact that the PSO algorithm is inherently parallel, since particles in the

swarm can largely be updated independently, making it exceptionally suitable for implementation on graphics processors. The main objective of the paper is to reduce the execution time of optimization problems compared to classic CPU implementations. The authors first provide an overview of the basic PSO algorithm, with a special emphasis on SPSO that uses a ring topology where the local best solution is chosen between the particle itself and its immediate neighbors. SPSO additionally introduces a constriction factor (χ) to improve the stability and convergence of the algorithm. Unlike the basic PSO, SPSO balances exploration and exploitation by combining a particle's own best experience with information from neighboring particles, reducing the risk of premature convergence. These improvements make SPSO more robust and effective for finding global optima, especially in complex, high-dimensional search spaces.

The paper explains the CUDA architecture and its suitability for parallel computing in detail. The authors highlight the importance of proper memory management, especially coalesced global memory access, which allows for maximum utilization of the GPU's memory bandwidth. Inefficient, non-coalesced memory access can significantly degrade the performance of the parallel implementation. The proposed GPU implementation of the SPSO algorithm consists of several CUDA kernel functions that perform the key stages of the algorithm in parallel: particle initialization, random number generation using the cuRAND library, fitness function calculation, updating pbest and lbest values, computing new velocities and positions, as well as determining the global best solution (gbest). It is particularly emphasized that all random numbers are generated directly on the GPU, avoiding expensive data transfers from the CPU to the GPU, which represents a significant improvement over earlier approaches.

To determine the global best solution, the atomicMin operation was used, which allows for correct parallel updating of shared data. Although CUDA does not directly support atomicMin for the float type, the problem was solved by converting to unsigned int, enabling efficient and secure reduction of fitness function values in a parallel environment. Experimental evaluation was conducted on seven standard benchmark functions using an NVIDIA GeForce GTX 980 GPU and an Intel Core i5 CPU, with various swarm sizes and problem dimensionalities. The results show that the GPU-SPSO implementation achieves a speedup of up to 46 times compared to the sequential CPU version, particularly with more complex functions and a higher number of iterations.

The work by Wang, Ho, Tu, and Hung [10] also focuses on accelerating PSO algorithms using GPU parallelization. Standard parallel approaches, such as parallel reduction, often suffer from an excessive number of memory accesses and thread synchronization issues, which limit their efficiency. To address these problems, the authors propose a Queue algorithm where each thread uses shared memory and atomic operations to conditionally update its results into a local queue within the block. Only the first thread of the block then searches the queue to find the local best result and stores it in auxiliary

arrays. This approach reduces complexity from $O(\log n)$ in reductions to a theoretical $O(1)$ on average.

Algorithm 1 The queue algorithm [10]

```

1: if fit > gbest_fit then
2:   unsigned const qIdx  $\leftarrow$  atomicAdd(&num, 1)
3:   bestFitQueue[qIdx]  $\leftarrow$  fit
4:   bestPosQueue[qIdx]  $\leftarrow$  pos
5: end if
6: __syncthreads();
7: if idx < particle_cnt && threadIdx.x == 0 then
8:   auxFit[blockIdx.x]  $\leftarrow$  INT_MIN
9:   auxPos[blockIdx.x]  $\leftarrow$  INT_MIN
10:  if num then
11:    for j in num do do
12:      if bestFitQueue[j] > bestFitQueue[0] then
13:        bestFitQueue[0]  $\leftarrow$  bestFitQueue[j]
14:        bestPosQueue[0]  $\leftarrow$  bestPosQueue[j]
15:      end if
16:    end for
17:    auxFit[blockIdx.x]  $\leftarrow$  bestFitQueue[0]
18:    auxPos[blockIdx.x]  $\leftarrow$  bestPosQueue[0]
19:  end if
20: end if

```

The paper further extends the algorithm into the Queue Lock algorithm, which introduces an atomic lock for secure updating of the global best solution among blocks. Each block waits for its turn using atomicCAS, then updates the global fitness and position, and a threadfence ensures that the changes are visible to all threads before the lock is released. This removes the need for executing a second kernel for the global best result and significantly reduces synchronization between blocks. With additional optimizations such as SoA (Structure of Arrays) layout for coalesced memory access, the use of constant memory for parameters, and shared memory for local queues, the proposed algorithm achieves execution hundreds of times faster than the serial CPU implementation and more than twice as fast as classical parallel methods.

III. IMPLEMENTATION

The methodology of this work primarily relies on a research approach and a comparative analysis of existing scientific solutions in the field of metaheuristic optimization. The first phase of the work involved researching relevant scientific literature and academic papers dealing with the application of the PSO algorithm within parallel computing architectures. A key part of the methodology represents the integration and upgrade of our previous work, where the existing sequential implementation served as the basic validation model. The development process was iterative; following the theoretical analysis of the selected scientific models, their algorithmic steps were mapped onto the CUDA programming framework, adapted to the specificities of the NVIDIA Tesla T4 graphics card. The choice of this hardware solution, based on the Turing architecture, was strategically dictated by the need for high

performance in data center environments (such as the Google Colab platform). With 2560 CUDA cores and 320 Tensor cores, the Tesla T4 enables massive parallel processing of thousands of particles simultaneously, while 16 GB of GDDR6 memory eliminates potential bottlenecks when working with complex search spaces. This approach allowed for a systematic transition from sequential logic to a model that maximizes the utilization of GPU resources, ensuring that the final implementation remains true to the theoretical foundations from the analyzed literature while providing significant speedup.

The first implemented solution is based on the SPSO model, specifically following the parallel architecture proposed by Hussain, Hattori, and Fujimoto in [9].

Unlike the basic PSO which relies on a Global Best (Gbest) update—often leading to premature convergence—this implementation utilizes a Ring Topology (Lbest model). In this structure, each particle communicates only with its immediate logical neighbors (indices $i - 1$ and $i + 1$). This neighborhood constraint is crucial for maintaining population diversity and ensuring the robustness of the search process in high-dimensional spaces. The algorithm is decomposed into four specialized kernels to ensure proper synchronization and data integrity between the evolutionary phases:

Algorithm 2 SPSO Parallel Architecture (CUDA Implementation)

Require: Population size N , Dimension D , Max Iterations $MaxIter$

Ensure: Global optimum found by the ring topology

```

1: {Initialization on Device}
2: Initialize particles' positions and velocities in GPU Global Memory
3: Initialize PRNG states for each CUDA thread
4: for  $t = 1$  to  $MaxIter$  do
5:   {Phase 1: Fitness Evaluation}
6:   call evaluate_kernel <<<(blocks, threads)>>>
7:   // Updates fitness and Pbest for each particle
8:   {Phase 2: Local Neighborhood Search (Ring Topology)}
9:   call lbest_kernel <<<(blocks, threads)>>>
10:  // Compares neighbors ( $i-1, i+1$ ) to find local optimum
11:  {Phase 3: Data Synchronization}
12:  call gather_lbest_pos <<<(blocks, threads)>>>
13:  // Fetches position data of identified Lbest particles
14:  {Phase 4: Kinematic Update}
15:  call update_kernel <<<(blocks, threads)>>>
16:  // Updates velocity/position using Coalesced Memory Access
17:  sync cudaDeviceSynchronize()
18: end for
19: return The best Pbest among all particles to Host

```

The second solution is based on the high-performance PSO framework by Wang et al. [10]. As shown in Algorithm 3, the architecture adapts to problem dimensionality through different execution modes.

Algorithm 3 High-Performance PSO - CUDA Implementation

Require: Population size N , Dimension D , Max Iterations $MaxIter$, Kernel Mode (Persistent/Standard)

Ensure: Global optimum $gBest$ synchronized via hardware-level atomic operations

```

1: {Device Resource Initialization}
2: call setup_kernel <<<(blocks, threads)>>> {Initialize cu-RAND states}
3: call init_kernel <<<(blocks, threads)>>> {Stochastic SoA initialization}
4: if  $D == 1$  and Kernel Mode == Persistent then
5:   {Low-Dimensional Optimization: Latency Minimization}
6:   call pso_queue_lock_persistent <<<(1, threads)>>>
7:   for  $t = 1$  to  $MaxIter$  do
8:     Update velocity and position within registers
9:     Evaluate fitness function
10:    atomicCAS with spin-lock to update  $gBest$ 
11:    __syncthreads() {Intra-block barrier for state consistency}
12:  end for
13: else
14:   {High-Dimensional Optimization (e.g., 120D)}
15:   for  $t = 1$  to  $MaxIter$  do
16:    {Phase 1: Kinematic Update and Intra-block Reduction}
17:    call pso_queue_kernel <<<(blocks, threads)>>>
18:    // Utilize __shared__ memory (bestFitQueue)
19:    // Intra-warp reduction via __shfl_down_sync
20:    // 64-bit Bit-packing (pack_fit_idx) for atomic update of auxFit
21:    {Phase 2: Global Aggregation}
22:    call reduce_global_best <<<(1, threads)>>>
23:    // Consolidate candidates from auxiliary buffers to final  $gBest$ 
24:    sync cudaDeviceSynchronize()
25:  end for
26: end if
27: return  $gBest$  from Device to Host

```

For low-dimensional problems (e.g., 1D), a persistent kernel executes the entire evolutionary loop (100,000 iterations) in a single launch. This approach eliminates kernel invocation latency and uses a hardware-level spin-lock for $gBest$ updates. In contrast, the 120D implementation employs a modular two-stage reduction to manage high memory load. It utilizes **__shared__** memory queues and a separate global reduction kernel to aggregate results from auxiliary buffers. To further optimize the 120D version, the **pso_ultra_kernel** introduces warp-level primitives. By using the **__shfl_down_sync** instruction, intra-warp aggregation is performed before accessing global memory, significantly reducing atomic unit pressure. Additionally, a 64-bit bit-packing technique (**pack_fit_idx**) was implemented.

This allows the fitness value and particle index to be stored in a single unsigned long long variable, enabling a single `atomicCAS` to update the global best state atomically and without race conditions. Data management follows a strict Structure of Arrays (SoA) pattern ($pos[d \cdot N + i]$) to ensure optimal memory coalescing.

IV. DISCUSSION OF RESULTS

The previously described SPSO implementation, based on the parallel architecture proposed by Hussain, Hattori, and Fujimoto [9], was evaluated through a direct comparison of execution times between the sequential CPU and parallel CUDA implementations. The experiments considered a high-dimensional optimization problem with 4,096 particles and 50 dimensions over 2,000 iterations using the Sphere benchmark function. To accurately measure execution time on the GPU side, `cudaEvent` markers were utilized, allowing for millisecond precision by measuring the elapsed time between the start and stop of the optimization loop. On the CPU side, the standard `clock()` function was employed to capture the total processing time of the serial implementation.

The measurement results indicate a significant advantage of the GPU architecture in mass-parallel swarm update operations. The following table summarizes the performance benchmarks obtained during the test. Due to the significant increase in computational complexity with higher dimensionality, the number of iterations was adjusted across experiments in order to maintain comparable runtime feasibility and ensure practical execution times on both CPU and GPU platforms.

TABLE I
PERFORMANCE METRICS AND SPEEDUP RATIO FOR THE SPSO IMPLEMENTATION (2,000 ITERATIONS)

Particles	Iterations	CPU Time (s)	GPU Time (s)	Speedup
4,096	2,000	19.6	0.33	59.4×

The calculated speedup factor, defined as the ratio $CPUtime/GPUtime$, reached 59.45x. This result confirms the high efficiency of the GPU in handling large datasets, achieving execution dozens of times faster than the traditional serial approach. While the sequential implementation represents the main bottleneck due to its iterative processing of each particle, the CUDA implementation successfully minimizes execution time by concurrently running kernels for evaluation, identifying local best positions (*lBest*), and updating particle kinematic parameters.

The substantial speedup observed in this study can be explained by a combination of advancements in hardware architecture and the characteristics of the execution environment. First, the reference model was originally evaluated on a Maxwell-based NVIDIA GTX 980, whereas the current experiments were performed using an NVIDIA Tesla T4 accelerator built on the newer Turing architecture. The Tesla T4 offers 2,560 CUDA cores along with improved instruction scheduling, which together provide significantly

higher throughput for continuous mathematical computations than older, consumer-grade GPUs. In addition, the experiments were conducted in a Google Colab environment, where the sequential implementation runs on a single virtualized CPU core of an Intel Xeon processor. This setup accentuates the performance gap, as a massively parallel, highly optimized GPU is compared against a virtualized CPU core that typically does not achieve the single-thread peak performance of dedicated high-end desktop processors, thereby further increasing the measured speedup.

The experimental results for the second implemented solution (1D Queue Lock SPSO implementation), based on the high-performance framework proposed by Wang et al. [10], reveal a significant performance advantage over the sequential CPU version. A key observation is the near-constant GPU execution time for swarm sizes up to 8,192 particles, demonstrating the GPU’s ability to hide memory latency and manage thread occupancy effectively. As the particle count increases to 131,072, the speedup ratio peaks at 127.25x. This high efficiency is primarily attributed to the persistent kernel strategy, which minimizes the overhead of global synchronization and kernel re-invocation, allowing the Queue Lock mechanism to handle *gBest* updates with minimal contention. These trends and specific performance gains are clearly visible in the data presented in Table II.

TABLE II
PERFORMANCE METRICS AND SPEEDUP RATIOS FOR THE 1D QUEUE LOCK SPSO IMPLEMENTATION (100,000 ITERATIONS)

Particles	Iterations	CPU Time (s)	GPU Time (s)	Speedup
128	100,000	0.385	0.1458	2.64×
256	100,000	0.825	0.2564	3.22×
512	100,000	1.503	0.2576	5.84×
1,024	100,000	3.042	0.2576	11.81×
2,048	100,000	6.277	0.2576	24.37×
4,096	100,000	12.410	0.2577	48.16×
8,192	100,000	23.850	0.2579	92.48×
16,384	100,000	47.355	0.4637	102.12×
32,768	100,000	94.629	0.9247	102.33×
65,536	100,000	200.536	1.6085	124.67×
131,072	100,000	378.671	2.9759	127.25×

the performance evaluation for the 120-dimensional PSO problem, utilizing the Queue-based reduction strategy, is presented in Table III. Transitioning to a high-dimensional search space significantly increases the computational intensity of the fitness function and particle updates, which is reflected in the CPU execution times. However, the GPU implementation exhibits exceptional scalability. A notable observation is the Speedup Ratio peaking at 229.42x for 32,768 particles.

The efficiency of the two-stage reduction—utilizing an intra-block shared memory queue followed by a global best update—is evident, as the GPU execution time remains remarkably low (under 0.2s) even as the swarm size grows to 8,192 particles. The speedup grows consistently with the increase in particle count, reaching its zenith in the range of 32k to 64k particles. Beyond this point, a slight decline in speedup is observed. This performance stagnation suggests

that the system is likely reaching a hardware saturation point. Given the high-dimensional (120D) nature of the problem, the immense volume of coordinate data processed per iteration indicates that global memory bandwidth, rather than computational power, becomes the primary limiting factor. This behavior is qualitatively consistent with the results presented in the reference study [10]. All detailed metrics described in this analysis are provided in Table III.

TABLE III
THE SPEEDUPS ACHIEVED BY THE QUEUE ALGORITHM FOR THE 120D PROBLEM

Particles	Iterations	CPU Time (s)	GPU Queue (s)	Speedup
128	5,000	2.392	0.4068	5.88×
256	4,000	3.543	0.2369	14.95×
512	3,000	5.305	0.1712	30.99×
1,024	2,000	7.078	0.1214	58.29×
2,048	2,000	14.214	0.1324	107.40×
4,096	1,500	21.593	0.1772	121.84×
8,192	1,000	29.494	0.1678	175.73×
16,384	1,000	59.125	0.2893	204.37×
32,768	1,000	128.349	0.5595	229.42×
65,536	1,000	237.933	1.1041	215.51×
131,072	800	379.820	1.7499	217.06×

Building upon the foundations of the reference implementation, we developed an enhanced version of the algorithm, hereafter referred to as Optimized cuPSO. This version introduces low-level hardware optimizations, including warp-level shuffle primitives to reduce atomic contention and a 64-bit bit-packing strategy for synchronized *gBest* updates. The results of this custom implementation are detailed in Table IV.

TABLE IV
COMPARISON OF CPU AND OPTIMIZED GPU cuPSO (120D)

Particles	Iterations	CPU Time (s)	GPU Opt (s)	Speedup
128	5,000	2.392	0.3198	7.48×
256	4,000	3.543	0.1520	23.31×
512	3,000	5.305	0.1182	44.89×
1,024	2,000	7.078	0.0878	80.58×
2,048	2,000	14.214	0.0932	152.53×
4,096	1,500	21.593	0.1389	155.51×
8,192	1,000	29.494	0.1371	215.21×
16,384	1,000	59.125	0.2508	235.78×
32,768	1,000	128.349	0.4762	269.51×
65,536	1,000	237.933	0.9611	247.55×
131,072	800	379.820	1.5605	243.39×

The data reveals a significant performance leap, with the speedup ratio reaching an impressive peak of 269.51x at 32,768 particles. By comparing these metrics to the standard 120D Queue algorithm, it is evident that our optimized approach consistently achieves lower GPU execution times across all swarm sizes. For instance, at 32,768 particles, the execution time was reduced from 0.5595s in the standard version to just 0.4762s in the optimized version. This improvement underscores the importance of minimizing warp-divergence and maximizing throughput via direct register communication. Similar to the previous tests, the system maintains high efficiency until the hardware reaches its global memory bandwidth

limit at 131,072 particles, yet it still sustains a massive speedup compared to the sequential CPU baseline.

The speedup comparison between the original Queue PSO implementation, based on the framework by Wang et al. [10], and our enhanced Optimized cuPSO version for the 120-dimensional problem is illustrated in Figure 1. The data reveals a consistent superiority of the optimized version, which maintains a higher speedup ratio across the entire range of swarm sizes due to the integration of warp-level shuffle primitives and 64-bit bit-packing techniques that minimize high-latency global memory operations. A notable peak performance of 269.51x speedup is achieved at 32,768 particles, significantly surpassing the original algorithm's maximum of 229.42x and demonstrating that the reduction of atomic contention within the warp effectively liberates GPU resources for fitness evaluations. While both implementations exhibit a stable upward trend and high scalability, the slight deceleration observed beyond the 32k particle threshold indicates that the hardware has reached a saturation point regarding memory bandwidth, which is expected given the massive data throughput required for a 120D search space. Furthermore, the optimized version shows remarkable efficiency even at lower loads, providing nearly triple the speedup of the original implementation at 2,048 particles, which confirms that our custom low-level optimizations successfully eliminated significant synchronization overhead present in the standard approach.

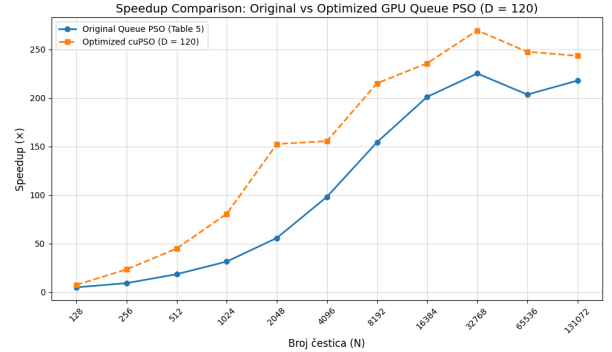


Fig. 1. Comparative Analysis of Speedup Ratios: Original Queue PSO vs. Optimized cuPSO (D=120)

The comparative GPU execution times for the 120-dimensional search space are illustrated in Figure 2, contrasting the performance of the original Queue PSO with our Optimized cuPSO implementation. The data reveals that the optimized version maintains consistently lower execution times across all tested swarm sizes, which is particularly evident in the range of 128 to 8,192 particles where the custom warp-level optimizations significantly reduce synchronization overhead. While both implementations follow a similar U-shaped efficiency curve—where execution time initially decreases as thread occupancy increases and subsequently rises as the hardware reaches its computational limit—the Optimized cuPSO demonstrates superior latency hiding. Even at the highest workload of 131,072 particles, the optimized

version completes the iterations in approximately 1.56 seconds compared to the 1.75 seconds required by the original implementation, confirming that the reduction of atomic contention directly translates to improved temporal efficiency.

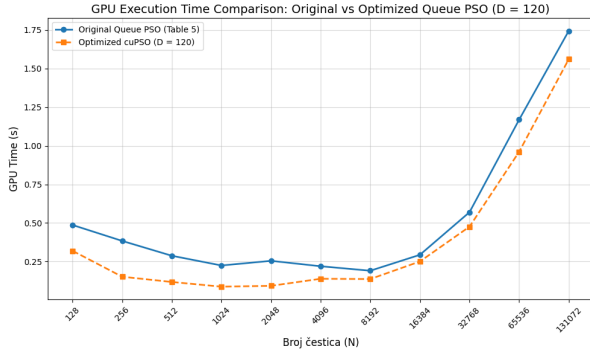


Fig. 2. GPU Execution Time Comparison: Original Queue PSO vs. Optimized cuPSO (D=120)

V. CONCLUSION

This research has demonstrated the significant performance advantages of leveraging GPU acceleration for PSO across diverse dimensional scales. By transitioning from sequential CPU-based execution to a massively parallel CUDA architecture, we have shown that PSO algorithms can achieve remarkable scalability, particularly as the complexity of the search space increases. The initial implementation of the Standard PSO with a Ring Topology provided a solid foundation, confirming that the GPU's ability to handle concurrent kinematic updates and fitness evaluations results in a substantial reduction in execution time compared to traditional serial approaches.

The experimental results further highlight the critical role of specialized synchronization mechanisms in high-dimensional optimization. The transition from a 1D problem to a 120-dimensional search space allowed the GPU to more effectively saturate its floating-point units and hide memory latency through massive parallelism. A key contribution of this work was the development of the Optimized cuPSO version, which pushed performance boundaries beyond the reference framework. By integrating warp-level shuffle primitives and 64-bit bit-packing techniques, we successfully minimized atomic contention and reduced the overhead associated with global memory access. This optimization strategy resulted in a peak speedup of 269.51x, significantly outperforming the standard queue-based reduction and demonstrating that low-level hardware tuning is essential for maximizing throughput on modern architectures like the NVIDIA Turing.

In conclusion, the findings of this study confirm that the primary bottlenecks in parallel PSO move from pure computational tasks to memory bandwidth and synchronization management as dimensionality grows. The adherence to the Structure of Arrays (SoA) pattern and the utilization of persistent kernels proved to be decisive factors in maintaining high occupancy and efficient data throughput. The developed

Optimized cuPSO framework provides a highly efficient solution for large-scale optimization tasks, offering a robust balance between search diversity and extreme computational performance. Future research may build upon these results by exploring multi-GPU scaling or the application of these optimized kernels to increasingly complex, non-linear objective functions in real-time environments.

REFERENCES

- [1] J. Kennedy and R. Eberhart, "Particle swarm optimization," in *Proceedings of ICNN'95 - International Conference on Neural Networks*. IEEE, 1995, pp. 1942–1948.
- [2] Y. Shi and R. Eberhart, "A modified particle swarm optimizer," in *1998 IEEE International Conference on Evolutionary Computation Proceedings*. IEEE, 1998, pp. 69–73.
- [3] M. Clerc and J. Kennedy, "The particle swarm - explosion, stability, and convergence in a multidimensional complex space," *IEEE Transactions on Evolutionary Computation*, vol. 6, no. 1, pp. 58–73, 2002.
- [4] R. Poli, J. Kennedy, and T. Blackwell, "Particle swarm optimization," *Swarm Intelligence*, vol. 1, no. 1, pp. 33–57, 2007.
- [5] A. P. Engelbrecht, *Fundamentals of Computational Swarm Intelligence*. John Wiley & Sons, 2007.
- [6] Y. Zhou and Y. Tan, "GPU-based parallel particle swarm optimization," *Proceedings of the 2011 IEEE Congress on Evolutionary Computation (CEC)*, pp. 1493–1500, 2011.
- [7] M. S. Nobile, P. Cazzaniga, D. Besozzi, R. Colombo, G. Mauri, and G. Pasi, "Fuzzy self-tuning PSO: A highly manageable parallel algorithm for efficient hybrid simulation of biological systems," *Swarm and Evolutionary Computation*, vol. 35, pp. 1–14, 2017.
- [8] L. Mussi, S. Cagnoni, and F. Daolio, "Evaluation of parallel particle swarm optimization algorithms within the cuda architecture," *Information Sciences*, vol. 181, no. 20, pp. 4642–4657, 2011.
- [9] M. M. Hussain, H. Hattori, and N. Fujimoto, "A CUDA implementation of the standard particle swarm optimization," in *Proceedings of the International Symposium on Computing and Networking*. Osaka, Japan: Osaka Prefecture University, 2013.
- [10] C.-C. Wang, C.-Y. Ho, C.-H. Tu, and S.-H. Hung, "High-performance particle swarm optimization on GPUs with CUDA," in *Proceedings of the 37th ACM/SIGAPP Symposium on Applied Computing (SAC '22)*. New York, NY, USA: Association for Computing Machinery, 2022, pp. 441–448. [Online]. Available: <https://doi.org/10.1145/3477314.3507142>